



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

- сообщения на inStudy

ИНТЕГРАЦИЯ КЛИЕНТСКОЙ И СЕРВЕРНОЙ ЧАСТЕЙ ВЕБ ПРИЛОЖЕНИЯ

2026 Г.



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

АРХИТЕКТУРА

ЗАЧЕМ НАМ ДВА ОТДЕЛЬНЫХ ПРИЛОЖЕНИЯ?



- Разделение ответственности:

- **Frontend:** Уровень представления (UI/UX).

Ответственность за рендеринг, сбор данных от пользователя и их первичную валидацию.

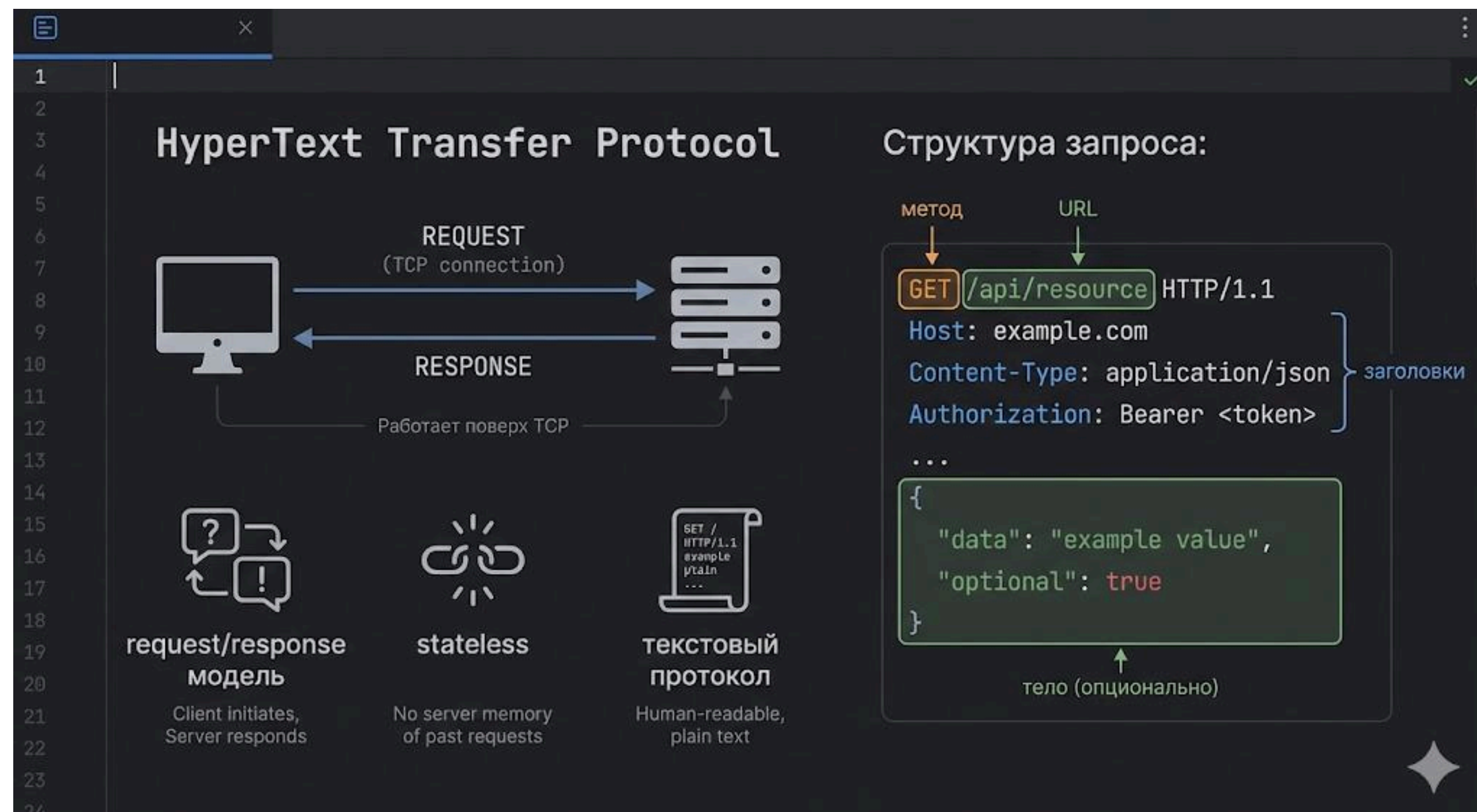
- **Backend:** Бизнес-логика, доступ к базе данных, авторизация и безопасность.

- Преимущества разделенной-архитектуры:

Масштабируемость, возможность использования одного API для разных платформ (Web, Mobile, IoT).

ПРОТОКОЛЫ И ФОРМАТЫ: HTTP И JSON

КАК КЛИЕНТ И СЕРВЕР ОБЩАЮТСЯ



- **Протокол HTTP:**

- Запрос (Request)
- Ответ (Response).

- **Анатомия запроса:**

- **Методы:**

- GET (чтение),
- POST (создание),
- PUT/PATCH (обновление),
- DELETE (удаление).

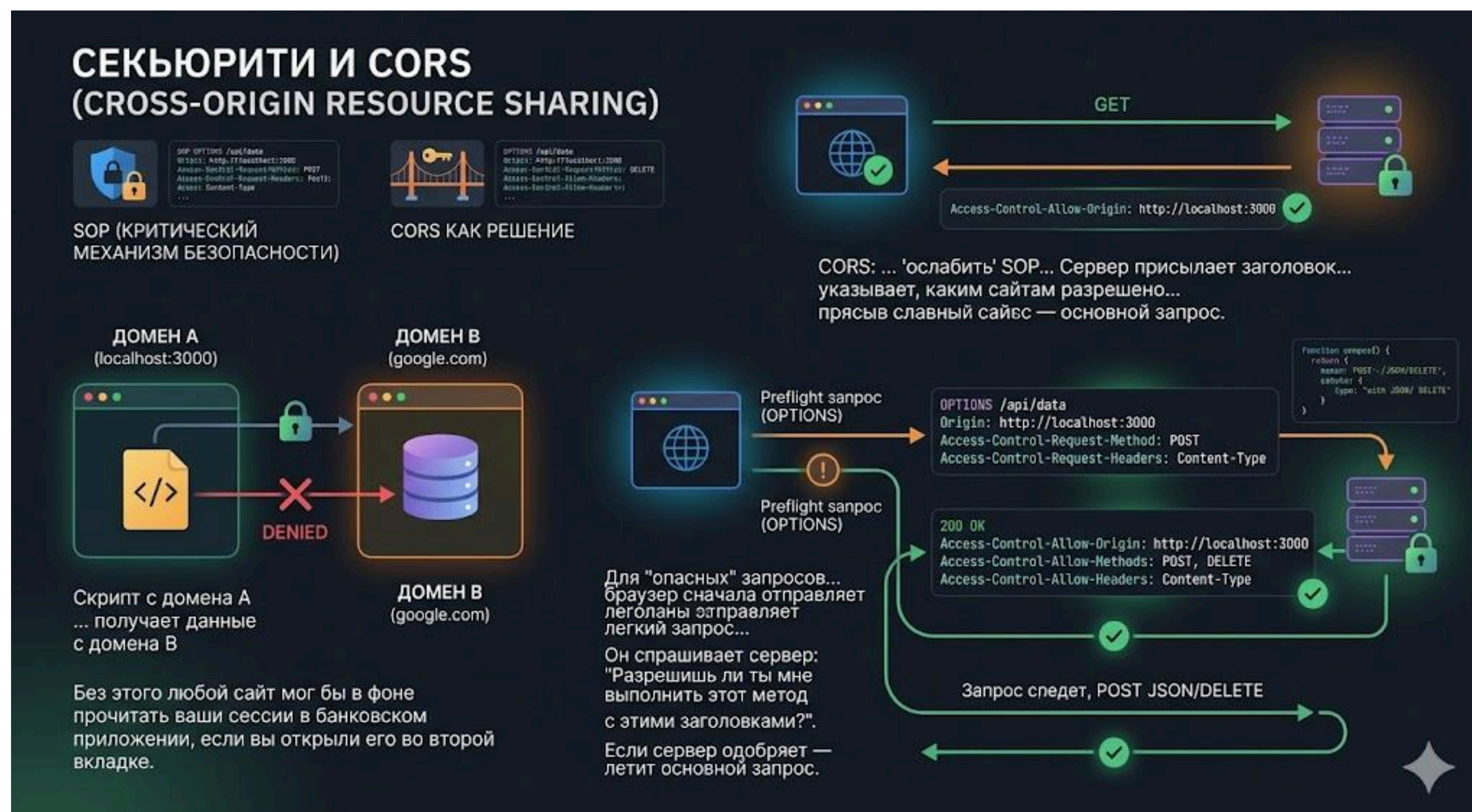
- **Тело запроса (Body):**

- Передача данных в формате JSON.

- **JSON (JavaScript Object Notation):** это стандарт для обмена данными в современном вебе.

БЕЗОПАСНОСТЬ И CORS

ВОПРОС ДОВЕРИЯ СЕРВЕРА К КЛИЕНТУ



- **Same-Origin Policy (SOP):**

- Критически важный механизм безопасности. Он запрещает скрипту с домена А (**localhost:3000**) получать данные с домена В (**localhost:4000**). Без этого любой сайт мог бы в фоне прочитать ваши сессии в банковском приложении, если вы открыли его во второй вкладке.

- **CORS как решение:**

- Это легальный способ "ослабить" SOP для **доверенных источников**. Сервер присылает заголовок **Access-Control-Allow-Origin**, в котором указывает, каким сайтам разрешено к нему обращаться.

ФУНКЦИЯ FETCH API И АСИНХРОННОСТЬ

FRONTEND. ПОЛУЧЕНИЕ И ОТПРАВКА ЗАПРОСОВ НА СЕРВЕР

```
const API_URL = 'http://localhost:3000';

async function apiFetch(endpoint, method = 'GET', body = null) {
  const options = {
    method,
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include' // <--- Чтобы сессия не пропала!
  };

  if (body) {
    options.body = JSON.stringify(body);
  }

  const res = await fetch(`${API_URL}${endpoint}`, options);
  return await res.json();
}
```

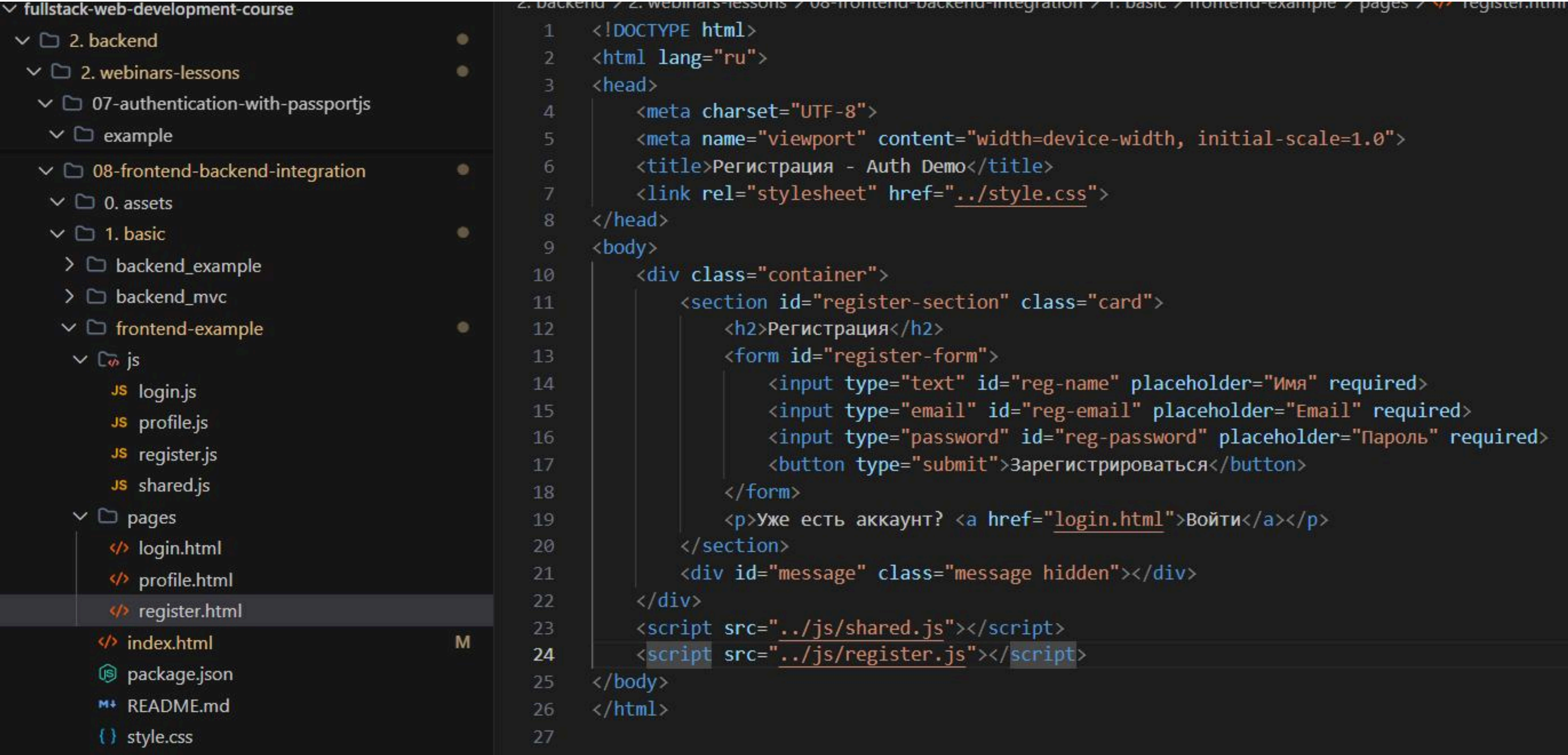
- **Fetch API:** Низкоуровневый интерфейс для асинхронных сетевых запросов. По умолчанию возвращает Promise.
- **Асинхронные операции и Async/Await:**
 - Сетевой запрос занимает время (мс или секунды). Мы не можем блокировать главный поток выполнения кода (иначе вкладка "зависнет").
 - **await** заставляет JS ждать завершения промиса (ответа от сервера) без блокировки браузера. Это делает асинхронный код визуально похожим на привычный синхронный.
- **Обработка ответов:** fetch не выдает ошибку, если сервер ответил 404 или 500. Нужно проверять `response.ok`. Также данные приходят в виде потока, поэтому их нужно распарсить: `await response.json()`.
- **Credentials (Режим 'include'):** По умолчанию fetch не отправляет куки (cookies) на сервер при кросс-доменных запросах. Чтобы сервер узнал залогиненного пользователя, нужно добавить в настройки `credentials: 'include'`.

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА

```
2. backend > 2. webinars-lessons > 08-frontend-backend-integration > 1. basic > frontend-example > index.html > html > body > script
1  <!DOCTYPE html>
2  <html lang="ru">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Auth Demo</title>
8      <link rel="stylesheet" href="style.css">
9      <script src="js/shared.js"></script>
10 </head>
11
12 <body>
13     <div class="container">
14         <section class="card">
15             <h2>Добро пожаловать</h2>
16             <p>Пример авторизации на Passport.js</p>
17             <div style="display: flex; flex-direction: column; gap: 10px; margin-top: 20px;">
18                 <button onclick="window.location.href='pages/login.html'">Войти</button>
19                 <button onclick="window.location.href='pages/register.html'"
20                     style="background-color: white; color: var(--primary); border: 2px solid var(--primary);">
21                     Регистрация
22                 </button>
23             </div>
24         </section>
25     </div>
26 </body>
27
28 </html>
```


ПРАКТИКА. СТРАНИЦА РЕГИСТРАЦИИ

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА



The screenshot shows a code editor with a dark theme. On the left is a file explorer for a project named 'fullstack-web-development-course'. The file structure is as follows:

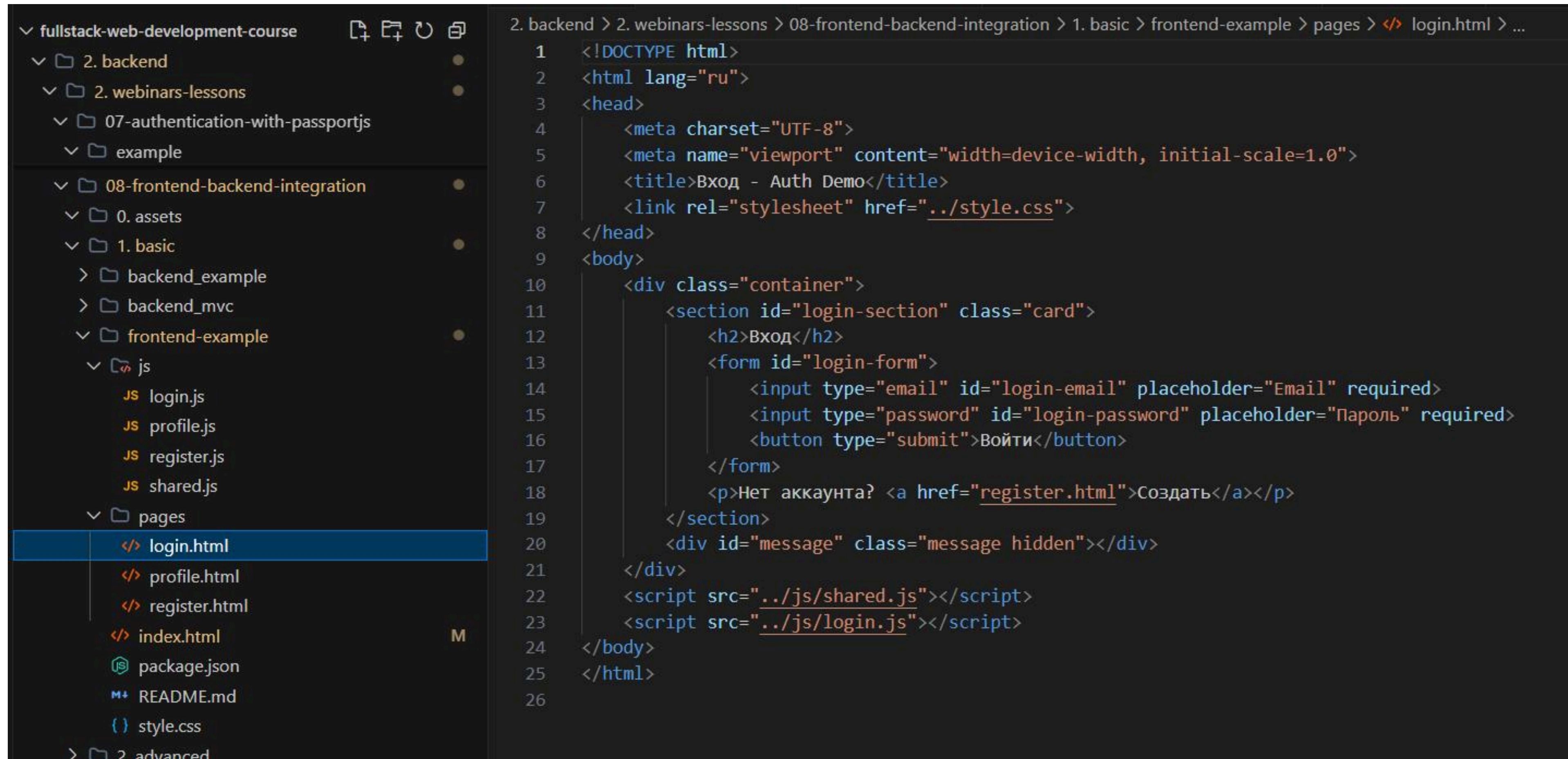
- fullstack-web-development-course
 - 2. backend
 - 2. webinars-lessons
 - 07-authentication-with-passportjs
 - example
 - 08-frontend-backend-integration
 - 0. assets
 - 1. basic
 - backend_example
 - backend_mvc
 - frontend-example
 - js
 - login.js
 - profile.js
 - register.js
 - shared.js
 - pages
 - login.html
 - profile.html
 - register.html
 - index.html
 - package.json
 - README.md
 - style.css

The file 'register.html' is selected and its content is displayed in the editor on the right. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Регистрация - Auth Demo</title>
7   <link rel="stylesheet" href="../style.css">
8 </head>
9 <body>
10   <div class="container">
11     <section id="register-section" class="card">
12       <h2>Регистрация</h2>
13       <form id="register-form">
14         <input type="text" id="reg-name" placeholder="Имя" required>
15         <input type="email" id="reg-email" placeholder="Email" required>
16         <input type="password" id="reg-password" placeholder="Пароль" required>
17         <button type="submit">Зарегистрироваться</button>
18       </form>
19       <p>Уже есть аккаунт? <a href="login.html">Войти</a></p>
20     </section>
21     <div id="message" class="message hidden"></div>
22   </div>
23   <script src="../js/shared.js"></script>
24   <script src="../js/register.js"></script>
25 </body>
26 </html>
27
```


ПРАКТИКА. СТРАНИЦА ВХОДА

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА

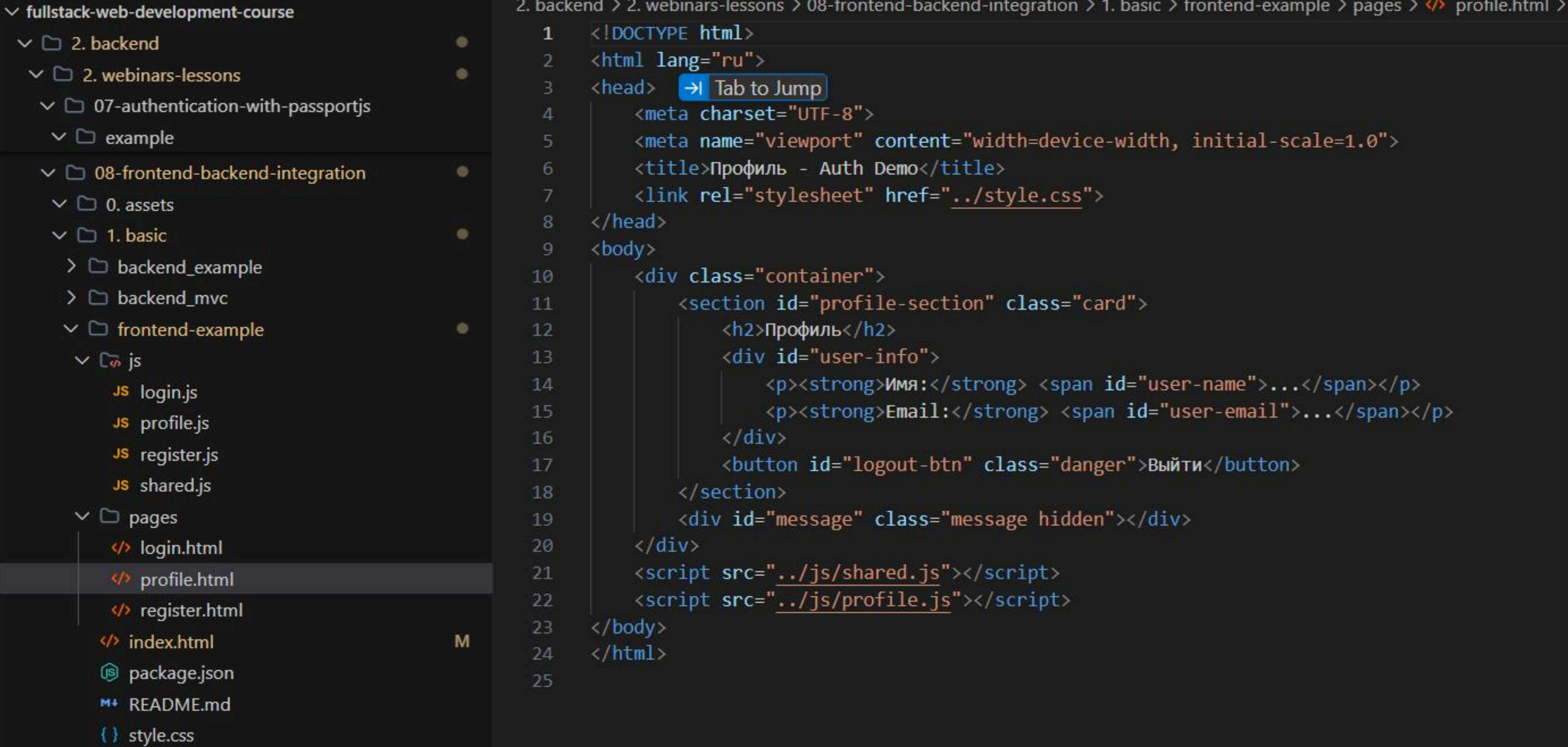


The screenshot shows a code editor with a dark theme. On the left is a file explorer showing the project structure. The selected file is `login.html` in the `pages` directory. The main editor area displays the HTML code for `login.html`, which includes a login form and a link to the register page.

```
2. backend > 2. webinars-lessons > 08-frontend-backend-integration > 1. basic > frontend-example > pages > login.html > ...  
1 <!DOCTYPE html>  
2 <html lang="ru">  
3 <head>  
4   <meta charset="UTF-8">  
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">  
6   <title>Вход - Auth Demo</title>  
7   <link rel="stylesheet" href="../style.css">  
8 </head>  
9 <body>  
10   <div class="container">  
11     <section id="login-section" class="card">  
12       <h2>Вход</h2>  
13       <form id="login-form">  
14         <input type="email" id="login-email" placeholder="Email" required>  
15         <input type="password" id="login-password" placeholder="Пароль" required>  
16         <button type="submit">Войти</button>  
17       </form>  
18       <p>Нет аккаунта? <a href="register.html">Создать</a></p>  
19     </section>  
20     <div id="message" class="message hidden"></div>  
21   </div>  
22   <script src="../js/shared.js"></script>  
23   <script src="../js/login.js"></script>  
24 </body>  
25 </html>  
26
```


ПРАКТИКА. СТРАНИЦА ПРОФИЛЯ

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА



```
2. backend > 2. webinars-lessons > 08-frontend-backend-integration > 1. basic > frontend-example > pages > profile.html >
1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Профиль - Auth Demo</title>
7      <link rel="stylesheet" href=" ../style.css">
8  </head>
9  <body>
10     <div class="container">
11         <section id="profile-section" class="card">
12             <h2>Профиль</h2>
13             <div id="user-info">
14                 <p><strong>Имя:</strong> <span id="user-name">...</span></p>
15                 <p><strong>Email:</strong> <span id="user-email">...</span></p>
16             </div>
17             <button id="logout-btn" class="danger">Выйти</button>
18         </section>
19         <div id="message" class="message hidden"></div>
20     </div>
21     <script src=" ../js/shared.js"></script>
22     <script src=" ../js/profile.js"></script>
23 </body>
24 </html>
25
```


ПРАКТИКА. ОБЩИЙ ШАБЛОН ФУНКЦИИ FETCH

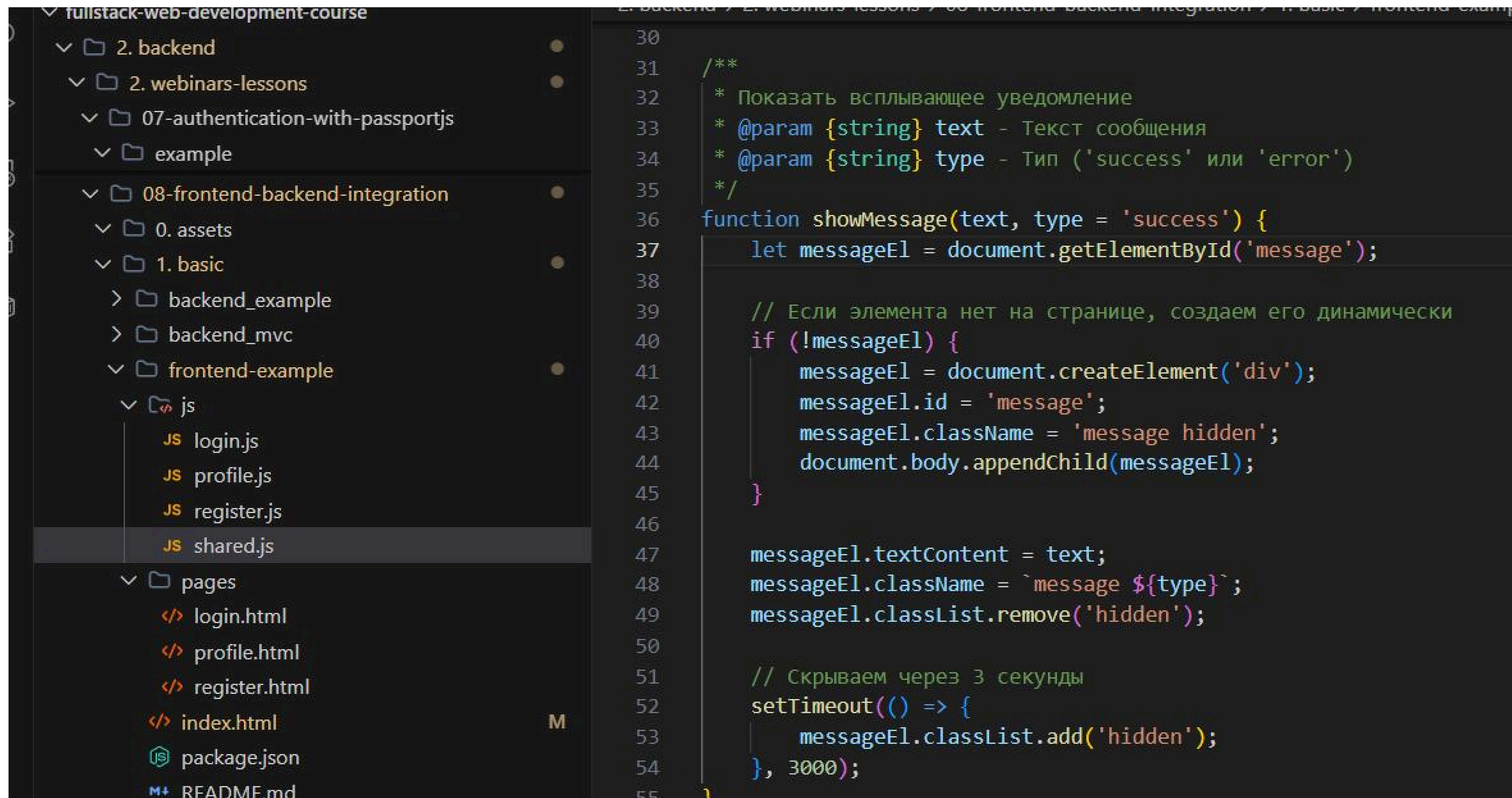
FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА

```
2. backend > 2. webinars-lessons > 08-frontend-backend-integration > 1. basic > frontend-example > js > JS shared.js > shd

1  const API_URL = 'http://localhost:3000';
2
3  /**
4   * Универсальный fetch с куками (для работы с сессиями)
5   * @param {string} endpoint - Путь (например, '/login')
6   * @param {string} method - HTTP метод (GET, POST, etc.)
7   * @param {object} body - Объект с данными для тела запроса
8   */
9  async function apiFetch(endpoint, method = 'GET', body = null) {
10     const options = {
11       method,
12       headers: {
13         'Content-Type': 'application/json'
14       },
15       credentials: 'include' // ВАЖНО: разрешает браузеру передавать куки (Session ID)
16     };
17
18     if (body) {
19       options.body = JSON.stringify(body);
20     }
21
22     try {
23       const res = await fetch(`${API_URL}${endpoint}`, options);
24       return await res.json();
25     } catch (err) {
26       console.error('API Error:', err);
27       return { status: 'error', message: 'Ошибка связи с сервером' };
28     }
29   }
30 }
```


ПРАКТИКА. ФУНКЦИЯ ПОКАЗА ВСПЛЫВАЮЩИХ СООБЩЕНИЙ

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА



```
30
31 /**
32  * Показать всплывающее уведомление
33  * @param {string} text - Текст сообщения
34  * @param {string} type - Тип ('success' или 'error')
35  */
36 function showMessage(text, type = 'success') {
37   let messageEl = document.getElementById('message');
38
39   // Если элемента нет на странице, создаем его динамически
40   if (!messageEl) {
41     messageEl = document.createElement('div');
42     messageEl.id = 'message';
43     messageEl.className = 'message hidden';
44     document.body.appendChild(messageEl);
45   }
46
47   messageEl.textContent = text;
48   messageEl.className = `message ${type}`;
49   messageEl.classList.remove('hidden');
50
51   // Скрываем через 3 секунды
52   setTimeout(() => {
53     messageEl.classList.add('hidden');
54   }, 3000);
55 }
```


ПРАКТИКА. ФУНКЦИЯ ОБРАБОТКИ РЕГИСТРАЦИИ

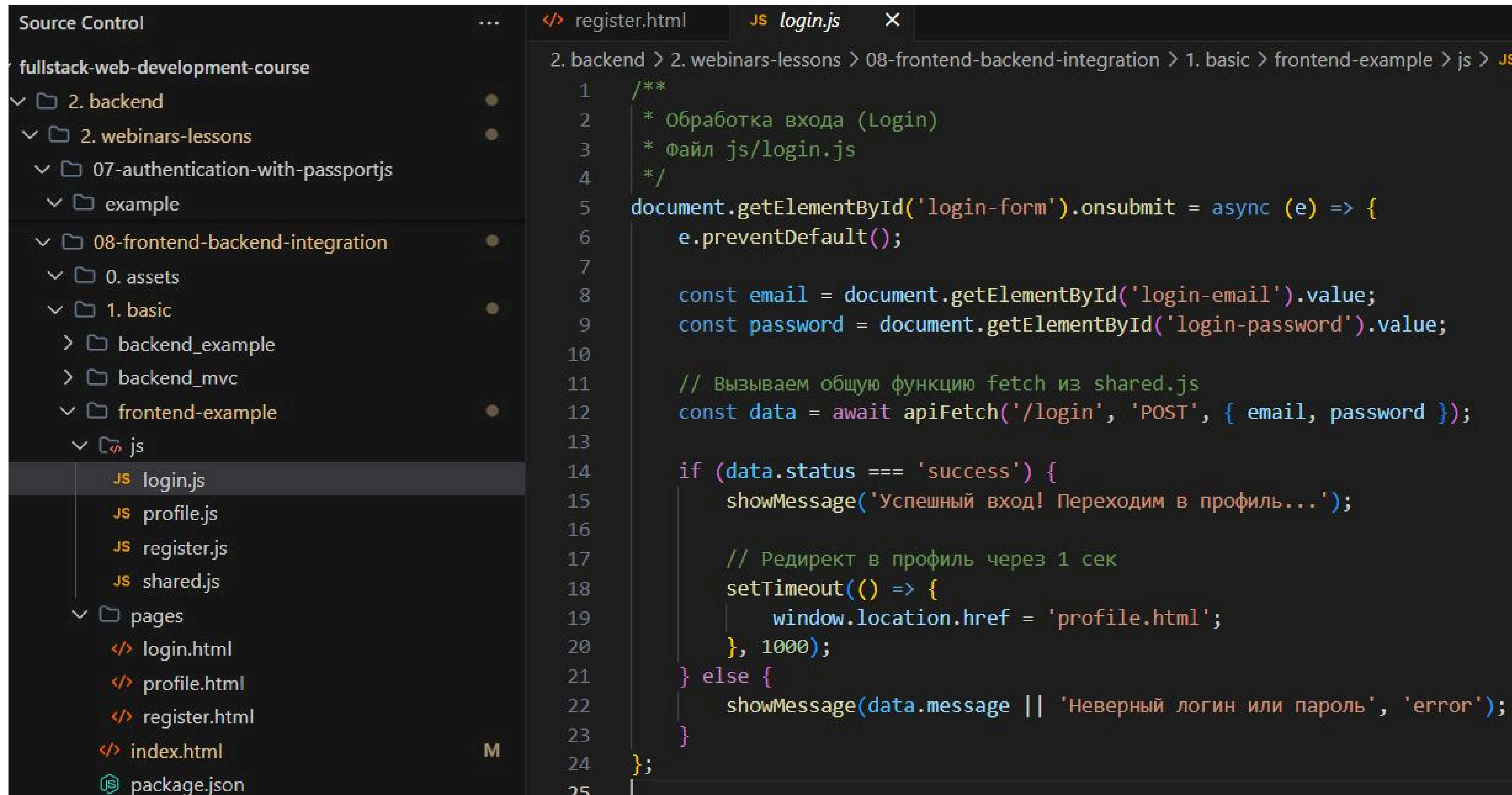
FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer on the left shows the project structure, with the file `register.js` selected under the `js` directory. The code editor on the right shows the content of `register.js`, which is a JavaScript file for handling registration. The code includes comments in Russian and JavaScript code for handling the registration form submission, including a fetch call to the `/register` endpoint and a redirect to the login page upon success.

```
1 /**
2  * Обработка регистрации (Register)
3  * Файл js/register.js
4  */
5 document.getElementById('register-form').onsubmit = async (e) => {
6   e.preventDefault();
7
8   const name = document.getElementById('reg-name').value;
9   const email = document.getElementById('reg-email').value;
10  const password = document.getElementById('reg-password').value;
11
12  // Вызываем общую функцию fetch из shared.js
13  const data = await apiFetch('/register', 'POST', { name, email, password });
14
15  if (data.status === 'success') {
16    showMessage('Регистрация успешна! Переходим ко входу...');
17
18    // Редирект ко входу через 1.5 сек
19    setTimeout(() => {
20      window.location.href = 'login.html';
21    }, 1500);
22  } else {
23    showMessage(data.message || 'Ошибка регистрации', 'error');
24  }
25 };
26
```


ПРАКТИКА. ФУНКЦИЯ ОБРАБОТКИ ВХОДА

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА

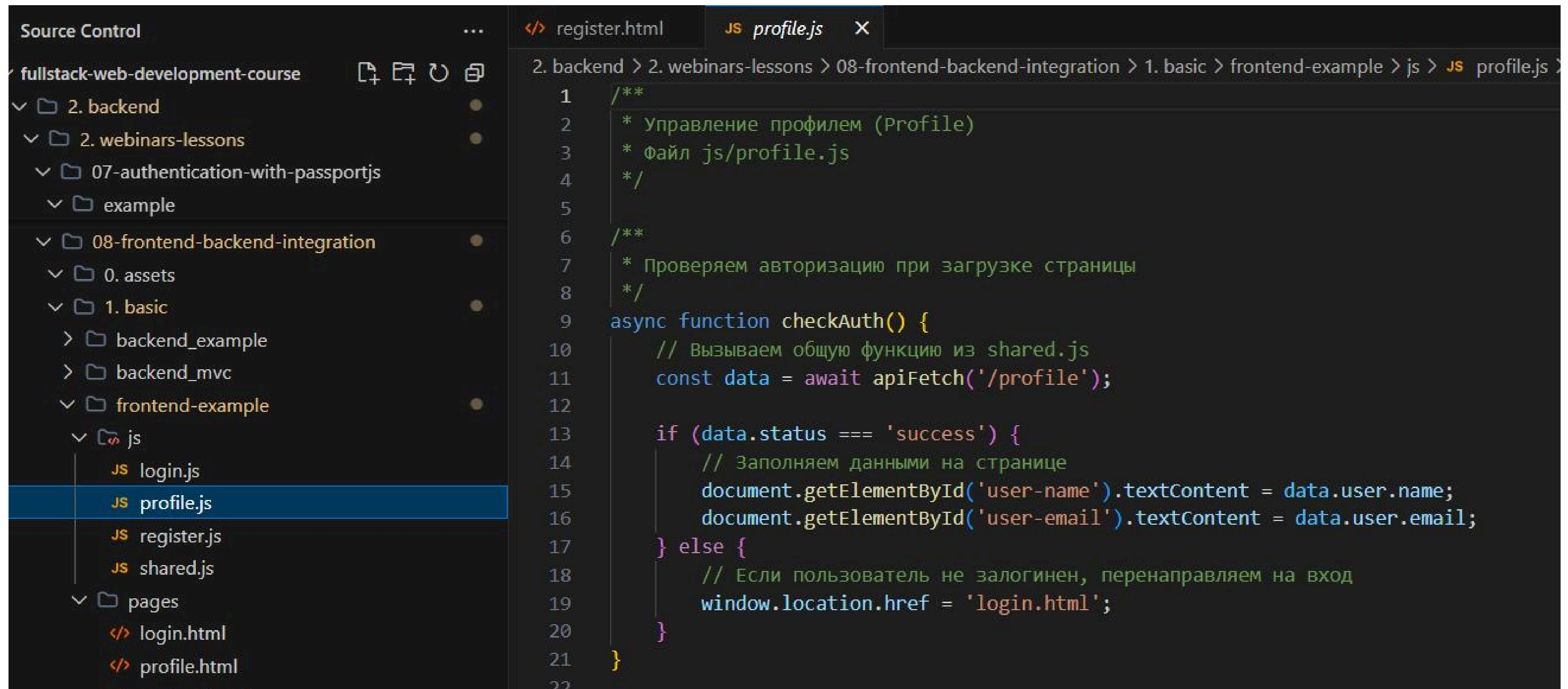


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure: fullstack-web-development-course > 2. backend > 2. webinars-lessons > 07-authentication-with-passportjs > example > 08-frontend-backend-integration > 1. basic > frontend-example > js > login.js. The code editor shows the implementation of the login function in login.js.

```
1  /**
2   * Обработка входа (Login)
3   * Файл js/login.js
4   */
5  document.getElementById('login-form').onsubmit = async (e) => {
6    e.preventDefault();
7
8    const email = document.getElementById('login-email').value;
9    const password = document.getElementById('login-password').value;
10
11    // Вызываем общую функцию fetch из shared.js
12    const data = await apiFetch('/login', 'POST', { email, password });
13
14    if (data.status === 'success') {
15      showMessage('Успешный вход! Переходим в профиль...');
16
17      // Редирект в профиль через 1 сек
18      setTimeout(() => {
19        window.location.href = 'profile.html';
20      }, 1000);
21    } else {
22      showMessage(data.message || 'Неверный логин или пароль', 'error');
23    }
24  };
25
```


ПРАКТИКА. ФУНКЦИЯ ПРОВЕРКИ ДОСТУПА

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА

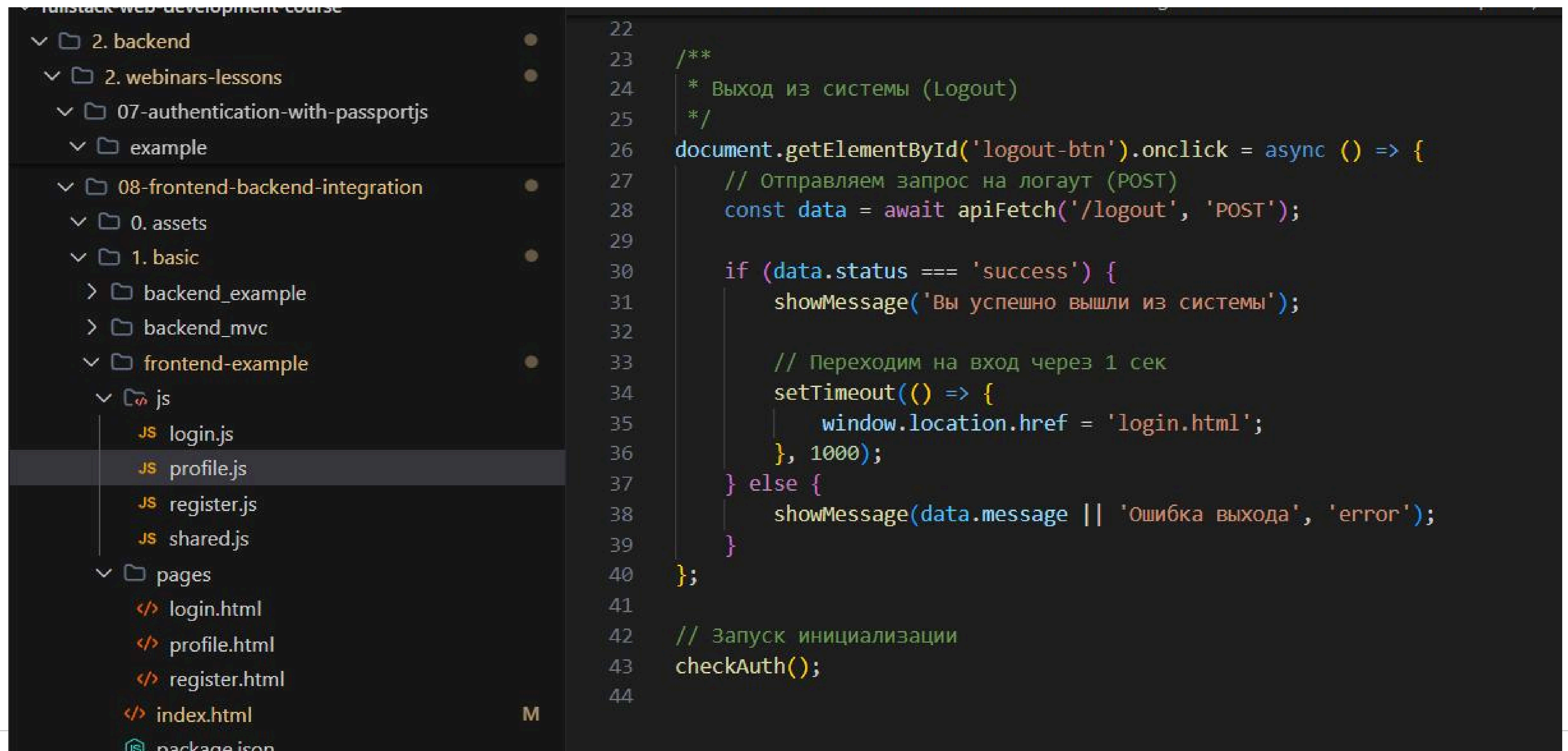


The screenshot shows the Visual Studio Code interface. On the left, the Source Control panel displays the file explorer for the project 'fullstack-web-development-course'. The file 'profile.js' is selected under the path '08-frontend-backend-integration > 1. basic > frontend-example > js'. The main editor window shows the code for 'profile.js'.

```
1  /**
2   * Управление профилем (Profile)
3   * Файл js/profile.js
4   */
5
6  /**
7   * Проверяем авторизацию при загрузке страницы
8   */
9  async function checkAuth() {
10     // Вызываем общую функцию из shared.js
11     const data = await apiFetch('/profile');
12
13     if (data.status === 'success') {
14         // Заполняем данными на странице
15         document.getElementById('user-name').textContent = data.user.name;
16         document.getElementById('user-email').textContent = data.user.email;
17     } else {
18         // Если пользователь не залогинен, перенаправляем на вход
19         window.location.href = 'login.html';
20     }
21 }
22
```


ПРАКТИКА. ФУНКЦИЯ ВЫХОДА

FRONTEND. РЕАЛИЗАЦИЯ ПРОСТОГО КЛИЕНТА ДЛЯ СЕРВЕРА



```
22
23 /**
24  * Выход из системы (Logout)
25  */
26 document.getElementById('logout-btn').onclick = async () => {
27     // Отправляем запрос на логат (POST)
28     const data = await apiFetch('/logout', 'POST');
29
30     if (data.status === 'success') {
31         showMessage('Вы успешно вышли из системы');
32
33         // Переходим на вход через 1 сек
34         setTimeout(() => {
35             window.location.href = 'login.html';
36         }, 1000);
37     } else {
38         showMessage(data.message || 'Ошибка выхода', 'error');
39     }
40 };
41
42 // Запуск инициализации
43 checkAuth();
44
```

ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ ПОЛЬЗОВАТЕЛЯ



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?